PRACTICAL FILE

Image Processing & Computer Vision

Lab Manual

Submitted in partial fulfilment of the requirement of the degree

of

BACHELOR OF TECHNOLOGY

CSE

To

K.R. Mangalam University

by

Nitesh Sharma

(2301010151)

B.Tech CSE Section - C

Department of Computer Science and Engineering

School of Engineering and Technology

K.R. Mangalam University, Gurugram - 122001, India

INDEX

No.	Experiment Title	Page No.
1	Image Acquisition and Display	3
2	Sampling and Quantization	5
3	Image Enhancement – Spatial Domain	7
4	Frequency Domain Filtering	9
5	Noise Model and Spatial Filtering	11
6	Periodic Noise Reduction	13
7	Geometric Transformations	15
8	Color Image Processing and Segmentation	17
9	Image Compression – Huffman and RLE Coding	19
10	Edge Detection and Segmentation	21
11	Morphological Operations	23
12	Threshold and Region-Based Segmentation	25
13	Shape Representation and Chain Code	27
14	CNN for Image Classification	29
15	Motion Estimation and Object Tracking	31
16	Face Recognition using Machine Learning	33

Experiment 1

Image Acquisition and Display

AIM	Write a program to acquire and display an image. Demonstrate the process of image sensing, digitization, and explain the components of an image acquisition system.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Image acquisition is the first step in digital image processing. It captures a real-world scene using a sensing device and converts it into a form suitable for computer processing.

The acquisition system consists of: (1) Illumination Source – provides energy (light, IR, X-ray) on the object; (2) Image Sensor – CCD or CMOS device converts reflected energy into electrical signals; (3) ADC – Analog-to-Digital Converter maps analog signals to discrete pixel values forming an M×N matrix of integers in [0, 255].

Algorithm

- Step 1: Import OpenCV, NumPy, and Matplotlib.
- Step 2: Read the image using cv2.imread().
- Step 3: Check if image loaded (not None).
- Step 4: Convert BGR to RGB; display using plt.imshow().
- Step 5: Print image shape, dtype, min/max pixel values.
- Step 6: Wait for keypress and release windows.

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread('sampleimage.jpg')

if image is None:
    print("Error: Image not found!")
else:
    # Convert BGR -> RGB for correct display
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(8, 5))
    plt.imshow(image_rgb)
    plt.title('Acquired Image')
    plt.axis('off')
    plt.show()

    # Print image properties
    print(f"Shape  : {image.shape}")
    print(f"Dtype  : {image.dtype}")
    print(f"Pixels : {image.shape[0] * image.shape[1]:,}")
    print(f"Min    : {image.min()}  Max: {image.max()}")
```

Output

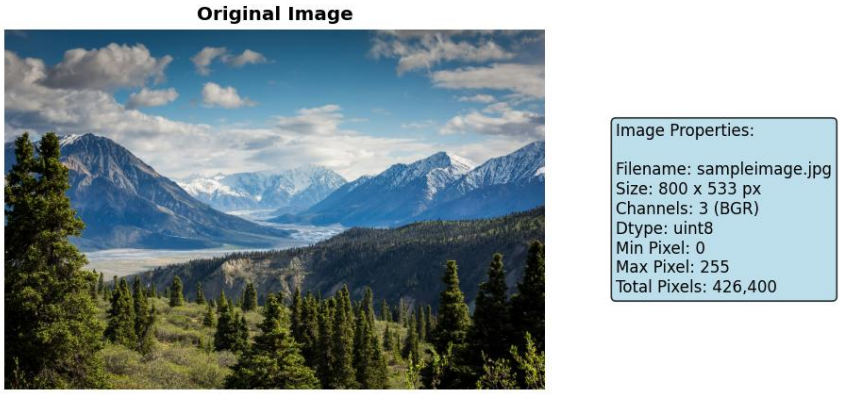

Fig 1.1: Acquired image with properties

Result & Conclusion

The program was successfully executed. The experiment demonstrates image acquisition and display using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 2

Sampling and Quantization

AIM	Perform sampling and quantization on a given image. Visualize the effects of different sampling rates and gray-level quantization on image quality.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Sampling defines spatial resolution — how many pixels represent the image. Reducing sampling rate causes pixelation (aliasing). Quantization defines the number of discrete grey levels (bit depth). With 8 bits, 256 levels exist. Reducing levels causes false contouring. Together, sampling and quantization determine the digital image's storage size and visual fidelity.

Algorithm

- Step 1: Load image and convert to grayscale.
- Step 2: Resize to three resolutions: 800×533, 400×267, 100×67.
- Step 3: Upscale all back to original size (nearest neighbor) for comparison.
- Step 4: Quantize to 256, 16, and 4 gray levels using integer division.
- Step 5: Display all results in a 2×3 subplot grid.
- Step 6: Observe pixelation and banding artifacts.

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
H, W  = gray.shape

# Spatial Sampling - resize and upscale back
fig, axes = plt.subplots(2, 3, figsize=(12, 7))
for i, (w, h) in enumerate([(W, H), (W//2, H//2), (W//8, H//8)]):
    small   = cv2.resize(gray, (w, h))
    display = cv2.resize(small, (W, H), interpolation=cv2.INTER_NEAREST)
    axes[0][i].imshow(display, cmap='gray')
    axes[0][i].set_title(f'{w}x{h}')

# Gray-Level Quantization
for i, levels in enumerate([256, 16, 4]):
    factor    = 256 // levels
    quantized = (gray // factor) * factor
    axes[1][i].imshow(quantized, cmap='gray')
    axes[1][i].set_title(f'{levels} Gray Levels')

plt.tight_layout()
plt.show()
```

Output

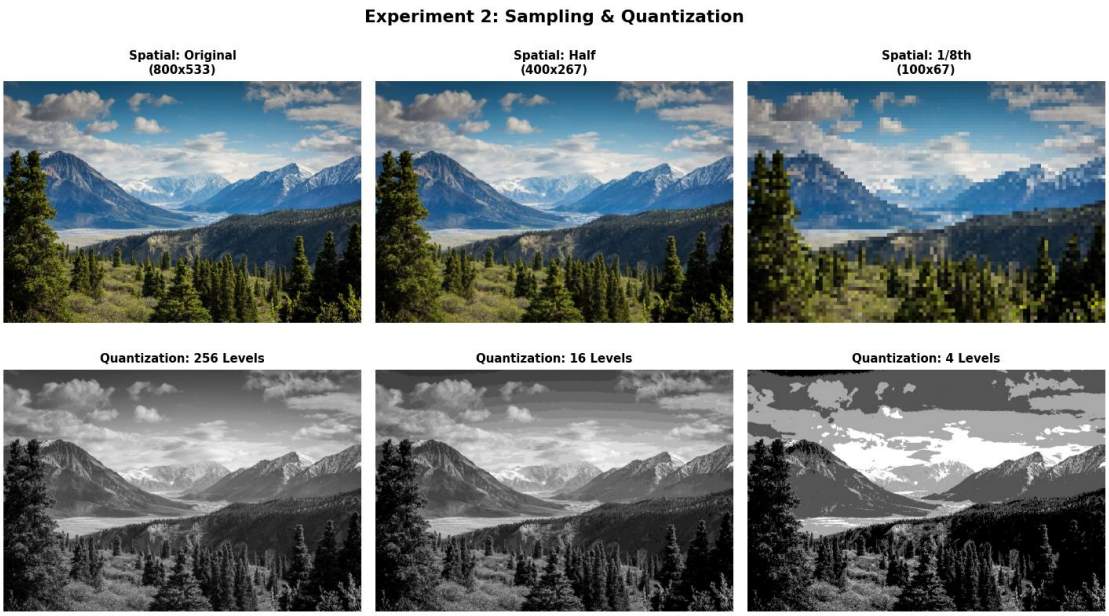

Fig 2.1: Spatial sampling (top) and gray-level quantization (bottom)

Result & Conclusion

The program was successfully executed. The experiment demonstrates sampling and quantization using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 3

Image Enhancement – Spatial Domain

AIM	Implement spatial domain image enhancement techniques: histogram equalization and contrast stretching. Analyze and compare their effects on image quality.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Image enhancement improves visual quality. Histogram Equalization redistributes pixel intensities to span the full dynamic range using the cumulative distribution function (CDF): s = T(r) = (L-1) × CDF(r). This maximizes contrast globally. Contrast Stretching linearly maps the pixel range [p2, p98] to [0, 255] using percentile clipping, avoiding saturation from extreme outliers.

Algorithm

- Step 1: Load image and convert to grayscale.
- Step 2: Plot the original histogram.
- Step 3: Apply cv2.equalizeHist() for histogram equalization.
- Step 4: Compute 2nd and 98th percentiles; apply linear stretch.
- Step 5: Plot histograms of all three versions.
- Step 6: Display original, equalized, and stretched images side by side.

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Histogram Equalization
hist_eq = cv2.equalizeHist(gray)

# Contrast Stretching (2nd-98th percentile)
p2, p98 = np.percentile(gray, 2), np.percentile(gray, 98)
stretched = np.clip(
    (gray.astype(float) - p2) / (p98 - p2) * 255, 0, 255
).astype(np.uint8)

# Display
fig, axes = plt.subplots(2, 3, figsize=(12, 7))
for i, (img, t) in enumerate(zip(
        [gray, hist_eq, stretched],
        ['Original', 'Hist. Equalized', 'Contrast Stretched'])):
    axes[0][i].imshow(img, cmap='gray');  axes[0][i].set_title(t)
    axes[1][i].hist(img.ravel(), 256, color='navy')
    axes[1][i].set_title(f'{t} Histogram')
plt.tight_layout(); plt.show()
```

Output

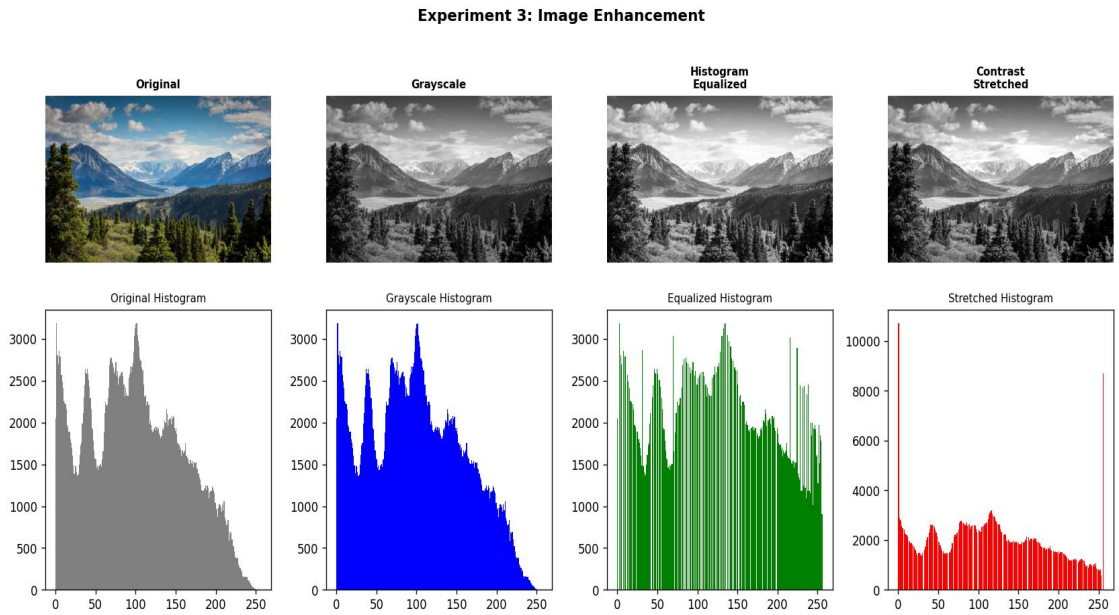

Fig 3.1: Enhanced images and their histograms

Result & Conclusion

The program was successfully executed. The experiment demonstrates image enhancement – spatial domain using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 4

Frequency Domain Filtering

AIM	Implement frequency domain filtering using the Fast Fourier Transform (FFT). Apply ideal low-pass and high-pass filters and observe their effects in the spatial domain.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

The 2D Fourier Transform decomposes an image into sinusoidal components of varying frequencies. Low frequencies (center of spectrum) represent smooth regions; high frequencies (periphery) represent edges and noise. Filtering in the frequency domain is equivalent to convolution in the spatial domain. A circular mask selects the desired frequency band. After filtering, IFFT reconstructs the spatial image.

Algorithm

- Step 1: Load image and convert to grayscale.
- Step 2: Compute 2D FFT using np.fft.fft2(); shift zero-freq to center.
- Step 3: Compute magnitude spectrum: 20·log(|F| + 1).
- Step 4: Create circular low-pass mask (radius=60); invert for high-pass.
- Step 5: Multiply spectrum by each mask.
- Step 6: Compute inverse FFT (IFFT) to recover filtered images.
- Step 7: Display original, spectrum, LPF output, HPF output.

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image  = cv2.imread('sampleimage.jpg')
gray   = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

f      = np.fft.fft2(gray)
fshift = np.fft.fftshift(f)
mag    = 20 * np.log(np.abs(fshift) + 1)

rows, cols = gray.shape
crow, ccol = rows // 2, cols // 2

# Low-Pass Mask (radius = 60)
mask_lp = np.zeros((rows, cols), np.uint8)
cv2.circle(mask_lp, (ccol, crow), 60, 1, -1)
mask_hp = 1 - mask_lp   # High-Pass is complement

def ifft_img(fs):
    return np.abs(np.fft.ifft2(np.fft.ifftshift(fs))).clip(0,255).astype(np.uint8)

img_lp = ifft_img(fshift * mask_lp)
img_hp = ifft_img(fshift * mask_hp)
```

Output

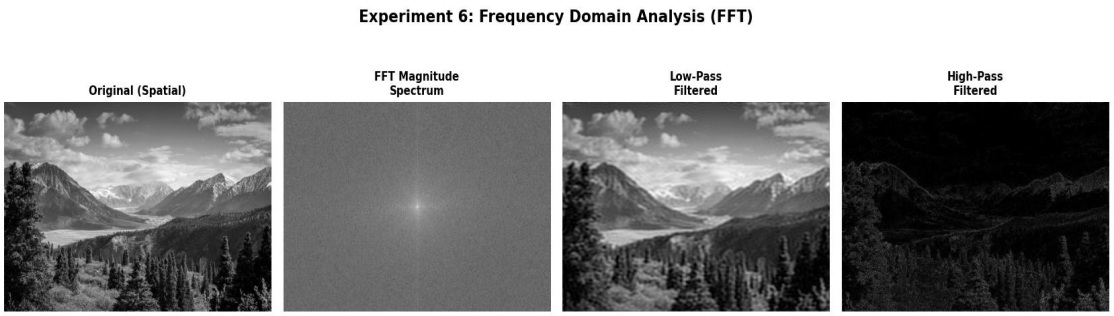

Fig 4.1: FFT magnitude spectrum, low-pass and high-pass filtered outputs

Result & Conclusion

The program was successfully executed. The experiment demonstrates frequency domain filtering using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 5

Noise Model and Spatial Filtering

AIM	Add Gaussian and salt-and-pepper noise to an image. Apply spatial filters (Average, Gaussian, Median, Bilateral) to remove noise and compare their effectiveness.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Noise degrades image quality during acquisition or transmission. Gaussian noise follows a normal distribution and is modeled as additive. Salt-and-pepper noise randomly sets pixels to 0 or 255. Linear filters (Average, Gaussian) smooth noise but blur edges. Median filter is non-linear and is highly effective for impulse noise while preserving edges. Bilateral filter smooths while preserving edges by weighing pixels by both spatial distance and intensity difference.

Algorithm

- Step 1: Load image; add Gaussian noise (σ=25) using np.random.normal().
- Step 2: Add salt-and-pepper noise (density=4%).
- Step 3: Apply Average filter (5×5) using cv2.blur().
- Step 4: Apply Gaussian filter (5×5) using cv2.GaussianBlur().
- Step 5: Apply Median filter (5×5) using cv2.medianBlur() on S&P noisy image.
- Step 6: Apply Bilateral filter using cv2.bilateralFilter(d=9, σ=75).
- Step 7: Display all results in a 2×4 grid.

Program Code

```
import cv2
import numpy as np

image = cv2.imread('sampleimage.jpg')

# Gaussian Noise
noise      = np.random.normal(0, 25, image.shape).astype(np.float32)
noisy_g    = np.clip(image.astype(np.float32) + noise, 0, 255).astype(np.uint8)

# Salt-and-Pepper Noise
sp = image.copy()
c1 = np.random.randint(0, sp.shape[0], 3000), np.random.randint(0, sp.shape[1],
3000)
sp[c1] = 255
c2 = np.random.randint(0, sp.shape[0], 3000), np.random.randint(0, sp.shape[1],
3000)
sp[c2] = 0

# Filters
avg      = cv2.blur(noisy_g, (5, 5))
gauss    = cv2.GaussianBlur(noisy_g, (5, 5), 0)
median   = cv2.medianBlur(sp, 5)
bilateral= cv2.bilateralFilter(noisy_g, 9, 75, 75)

sharp_k  = np.array([[-1,-1,-1],[-1,9,-1],[-1,-1,-1]])
sharpened= cv2.filter2D(image, -1, sharp_k)
```

Output

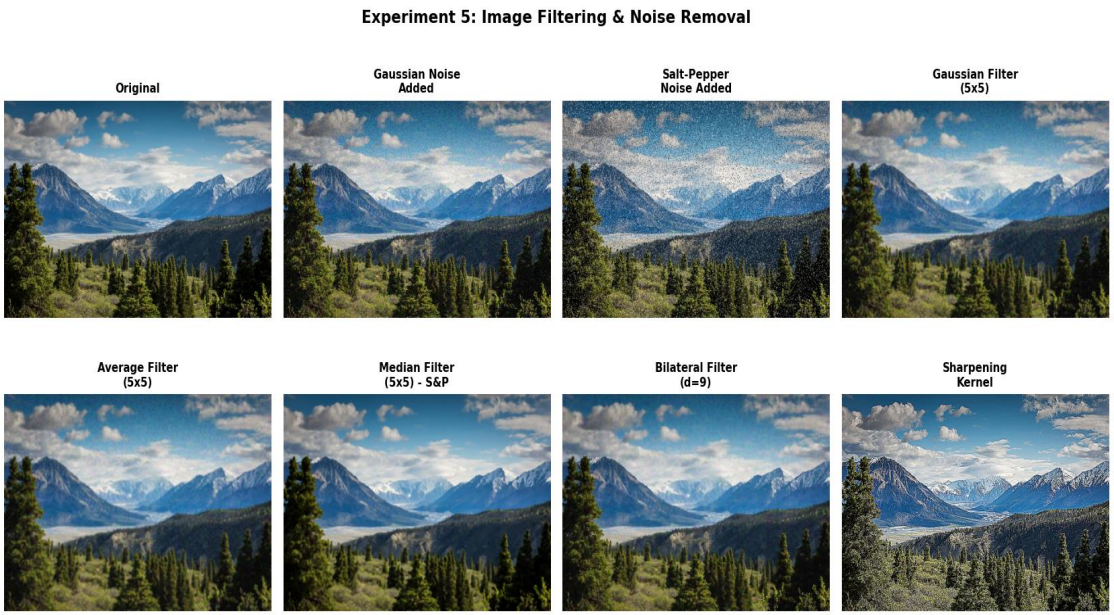

Fig 5.1: Noisy images and spatial filter results

Result & Conclusion

The program was successfully executed. The experiment demonstrates noise model and spatial filtering using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 6

Periodic Noise Reduction

AIM	Simulate periodic noise in an image and remove it using a notch filter applied in the frequency domain. Visualize the noise signature in the FFT spectrum.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Periodic noise appears as regular patterns caused by electrical or mechanical interference. In the frequency domain, periodic noise manifests as distinct bright spikes in the FFT magnitude spectrum at specific frequencies. A notch filter selectively suppresses these spike frequencies by zeroing out small circular regions around them, without affecting the rest of the spectrum. The filtered image is recovered by computing the Inverse FFT.

Algorithm

- Step 1: Load image and convert to grayscale.
- Step 2: Add sinusoidal periodic noise: $f(x,y) \mathrel{+}= 30 \cdot \sin(2\pi \cdot y/8)$.
- Step 3: Compute FFT and shift; visualize magnitude spectrum.
- Step 4: Identify spike locations in the spectrum.
- Step 5: Create notch mask — zero circles at spike locations.
- Step 6: Multiply spectrum by notch mask; compute IFFT.
- Step 7: Display original, noisy, spectrum, and restored images.

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
rows, cols = gray.shape

# Add Periodic Noise (sinusoidal)
noisy = gray.astype(np.float32).copy()
for i in range(rows):
    noisy[i] += 30 * np.sin(2 * np.pi * i / 8)
noisy = np.clip(noisy, 0, 255).astype(np.uint8)

# FFT and Notch Filter
f      = np.fft.fftshift(np.fft.fft2(noisy.astype(np.float32)))
mag    = 20 * np.log(np.abs(f) + 1)

mask = np.ones((rows, cols))
for r in [rows // 8, rows - rows // 8]:   # Spike locations
    cv2.circle(mask, (cols // 2, r), 10, 0, -1)

filtered = np.abs(np.fft.ifft2(
    np.fft.ifftshift(f * mask))).clip(0, 255).astype(np.uint8)
```

Output

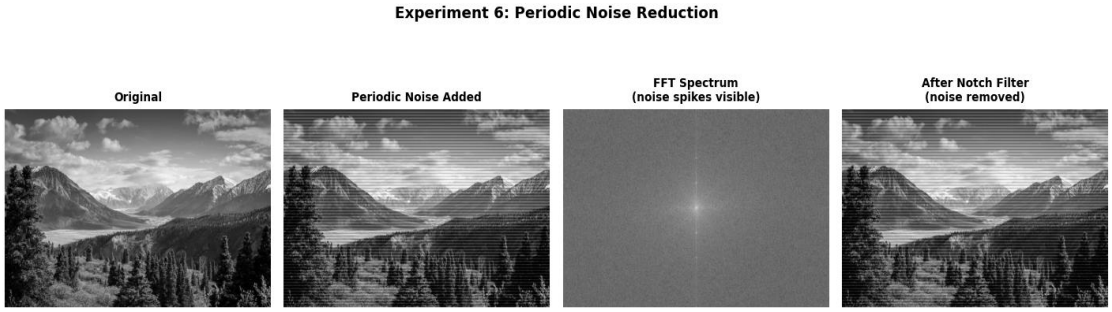

Fig 6.1: Periodic noise added, FFT spectrum with spikes, and notch-filtered result

Result & Conclusion

The program was successfully executed. The experiment demonstrates periodic noise reduction using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 7

Geometric Transformations

AIM	Implement geometric transformations: translation, rotation, scaling, affine (shear), and perspective transform. Observe spatial layout changes after each transformation.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Geometric transformations map pixel coordinates from input to output. Translation: [x',y'] = [x+tx, y+ty]. Rotation uses a 2×2 rotation matrix about a pivot. Scaling changes spatial dimensions. Affine Transform preserves parallelism and allows translation, rotation, scaling, and shear — defined by 3 point pairs. Perspective Transform (homography) allows non-parallel projections for viewpoint correction — defined by 4 point pairs.

Algorithm

- Step 1: Load image; get dimensions (H, W).
- Step 2: Translation: build M = [[1,0,tx],[0,1,ty]]; apply cv2.warpAffine().
- Step 3: Rotation: M = cv2.getRotationMatrix2D(center, 15°, 1.0).
- Step 4: Scaling: cv2.resize() at 0.5×.
- Step 5: Affine shear: compute M from 3 point pairs.
- Step 6: Perspective: compute M from 4 corner point pairs; cv2.warpPerspective().
- Step 7: Display all in 2×3 grid.

Program Code

```
import cv2
import numpy as np

image = cv2.imread('sampleimage.jpg')
h, w  = image.shape[:2]

# Translation
M_t = np.float32([[1, 0, 50], [0, 1, 30]])
translated = cv2.warpAffine(image, M_t, (w, h))

# Rotation (15 degrees about center)
M_r  = cv2.getRotationMatrix2D((w//2, h//2), 15, 1.0)
rotated = cv2.warpAffine(image, M_r, (w, h))

# Scaling
scaled = cv2.resize(image, (w//2, h//2))

# Affine (Shear)
pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])
affine = cv2.warpAffine(image, cv2.getAffineTransform(pts1,pts2), (w,h))

# Perspective
src = np.float32([[0,0],[w,0],[0,h],[w,h]])
dst = np.float32([[50,30],[w-50,0],[0,h],[w,h-30]])
perspective = cv2.warpPerspective(image,
              cv2.getPerspectiveTransform(src,dst), (w,h))
```

Output

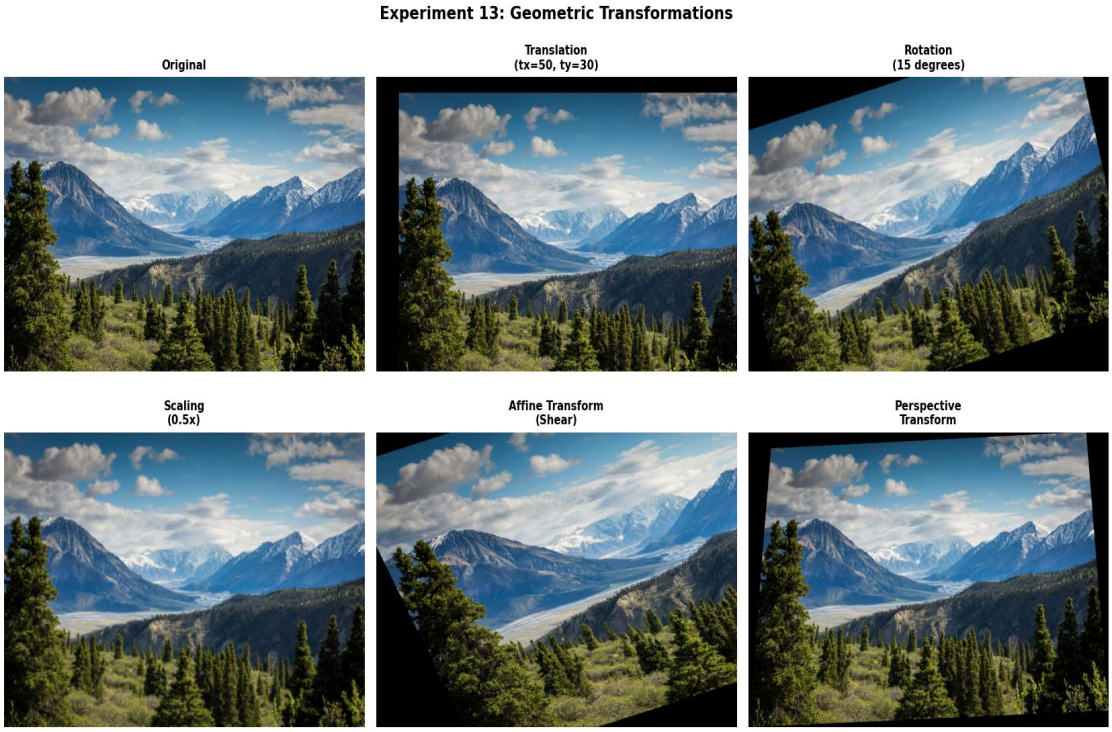

Fig 7.1: Translation, rotation, scaling, affine, and perspective transforms

Result & Conclusion

The program was successfully executed. The experiment demonstrates geometric transformations using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 8

Color Image Processing and Segmentation

AIM	Implement color image processing including RGB channel extraction, color space conversions (HSV, LAB), and color-based segmentation using HSV thresholding and K-means clustering.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Color models define color representation. RGB uses three additive channels (Red, Green, Blue). HSV (Hue, Saturation, Value) separates color from brightness — ideal for color-based segmentation. LAB separates luminance from chrominance and is perceptually uniform. K-means clustering groups pixels with similar color into K clusters by minimizing within-cluster variance using the Euclidean distance in color space.

Algorithm

- Step 1: Load color image; split into R, G, B channels.
- Step 2: Convert to HSV and LAB color spaces.
- Step 3: Create HSV range masks for sky (blue) and vegetation (green).
- Step 4: Reshape image pixels to (N,3); run cv2.kmeans(K=4).
- Step 5: Apply Otsu's threshold on grayscale.
- Step 6: Display RGB channels, color spaces, and all segmentation results.
- Step 7: Analyze how each method separates scene regions.

Program Code

```
import cv2
import numpy as np

image   = cv2.imread('sampleimage.jpg')
img_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray    = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
hsv     = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Color-based Segmentation (HSV)
sky_mask   = cv2.inRange(hsv, np.array([100,50,100]), np.array([140,255,255]))
green_mask = cv2.inRange(hsv, np.array([35,40,40]),   np.array([85,255,255]))

seg = np.zeros_like(img_rgb)
seg[sky_mask   > 0] = [135, 206, 235]
seg[green_mask > 0] = [34,  139,  34]

# K-Means Segmentation (K=4)
pixels = img_rgb.reshape(-1, 3).astype(np.float32)
_, labels, centers = cv2.kmeans(pixels, 4, None,
    (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0),
    10, cv2.KMEANS_RANDOM_CENTERS)
kmeans_img = np.uint8(centers)[labels.flatten()].reshape(img_rgb.shape)
```

Output

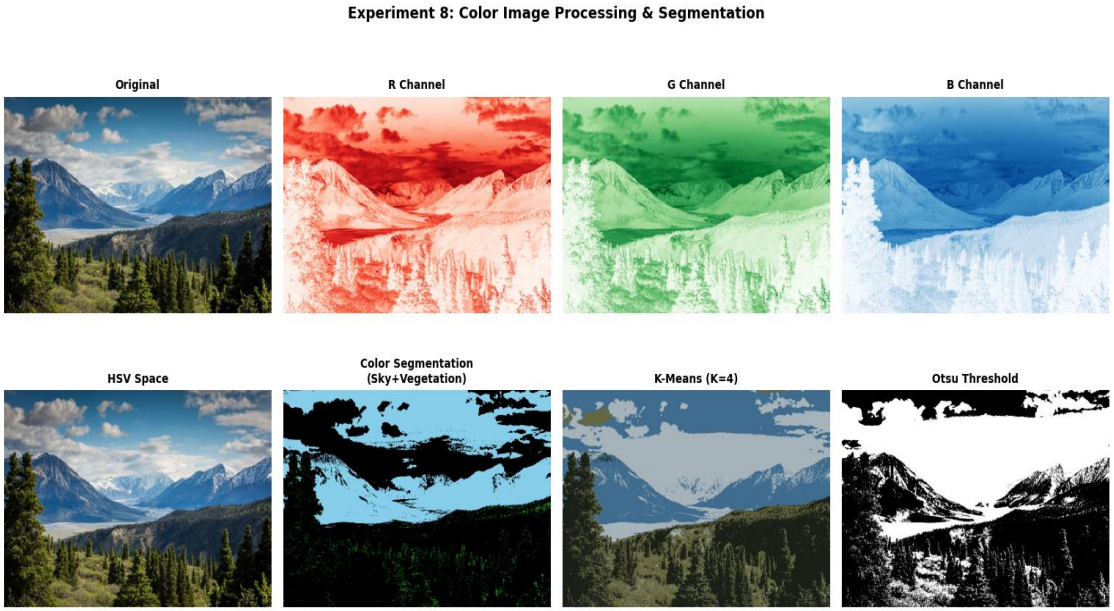

Fig 8.1: RGB channels, HSV segmentation, and K-means color segmentation

Result & Conclusion

The program was successfully executed. The experiment demonstrates color image processing and segmentation using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 9

Image Compression – Huffman and RLE Coding

AIM	Demonstrate image compression using DCT-based JPEG (simulating Huffman coding) at multiple quality levels. Measure compression ratios and observe quality trade-offs.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Image compression reduces storage size. Run-Length Encoding (RLE) stores consecutive repeated values as (value, count) pairs — efficient for binary images. Huffman Coding assigns shorter bit codes to more frequent symbols, achieving entropy-optimal lossless compression. JPEG uses DCT to transform 8×8 blocks to frequency domain, quantizes coefficients (lossy), then applies Huffman coding (lossless). Higher quantization = smaller file but lower quality.

Algorithm

- Step 1: Load image; extract a 64×64 patch for DCT visualization.
- Step 2: Compute 2D DCT using scipy.fftpack.dct(); visualize coefficients.
- Step 3: Encode full image at JPEG quality 90 using cv2.imencode().
- Step 4: Repeat for quality 30 and quality 5.
- Step 5: Decode each compressed version with cv2.imdecode().
- Step 6: Record file sizes and compute compression ratios.
- Step 7: Display original vs all compressed versions.

Program Code

```
import cv2
import numpy as np
from scipy.fftpack import dct, idct

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 2D DCT on a 64x64 patch
def dct2(x):  return dct(dct(x.T, norm='ortho').T, norm='ortho')
def idct2(x): return idct(idct(x.T, norm='ortho').T, norm='ortho')

patch    = gray[:64, :64].astype(np.float32)
dct_coef = dct2(patch)   # Frequency-domain representation

# JPEG compression at multiple quality levels
results = {}
for q in [90, 30, 5]:
    _, enc = cv2.imencode('.jpg', image, [cv2.IMWRITE_JPEG_QUALITY, q])
    results[q] = {'img': cv2.imdecode(enc, 1), 'kb': len(enc) // 1024}

# Print compression ratios
orig_kb = image.nbytes // 1024
for q, r in results.items():
    ratio = orig_kb / max(r['kb'], 1)
    print(f"Q={q:3d}: {r['kb']:4d} KB  (ratio {ratio:.1f}:1)")
```

Output

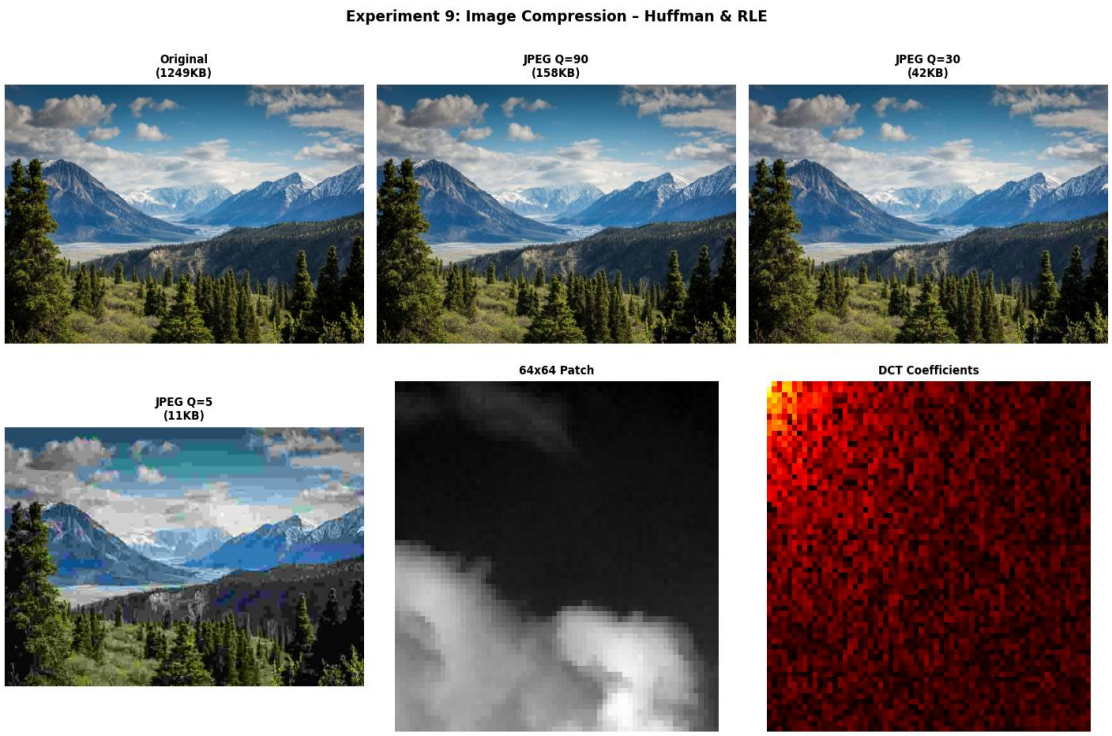

Fig 9.1: JPEG compression at Q=90, Q=30, Q=5 and DCT coefficient map

Result & Conclusion

The program was successfully executed. The experiment demonstrates image compression – huffman and rle coding using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 10

Edge Detection and Segmentation

AIM	Implement and compare edge detection operators: Sobel, Prewitt, Roberts Cross, Laplacian, and Canny. Analyze noise sensitivity and edge localization quality of each method.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Edge detection finds boundaries between regions. Gradient operators (Sobel, Prewitt, Roberts) compute first-order derivatives — edges are local maxima of the gradient magnitude. Laplacian computes second-order derivatives — edges are zero crossings. Canny is a multi-stage method: Gaussian smoothing → gradient magnitude → non-maximum suppression → double thresholding → hysteresis tracking. Canny provides the best combination of noise immunity, thin edges, and accurate localization.

Algorithm

- Step 1: Load image and convert to grayscale.
- Step 2: Sobel: cv2.Sobel() in X and Y; compute magnitude.
- Step 3: Prewitt: apply custom 3×3 kernels via cv2.filter2D().
- Step 4: Roberts: apply 2×2 diagonal kernels.
- Step 5: Laplacian: cv2.Laplacian(); take absolute value.
- Step 6: Canny: cv2.Canny(T1=50, T2=150).
- Step 7: Display all 6 results in 2×3 grid.

Program Code

```
import cv2
import numpy as np

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Sobel
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = np.uint8(255 * np.hypot(sx, sy) / np.hypot(sx, sy).max())

# Prewitt
kx = np.array([[1,1,1],[0,0,0],[-1,-1,-1]], np.float32)
ky = np.array([[-1,0,1],[-1,0,1],[-1,0,1]], np.float32)
prewitt = np.uint8(255 * np.hypot(
    cv2.filter2D(gray.astype(float),-1,kx),
    cv2.filter2D(gray.astype(float),-1,ky)) / 255)

# Roberts
rx = np.array([[1,0],[0,-1]], np.float32)
ry = np.array([[0,1],[-1,0]], np.float32)
roberts = np.uint8(255 * np.hypot(
    cv2.filter2D(gray.astype(float),-1,rx),
    cv2.filter2D(gray.astype(float),-1,ry)) / 255)

# Laplacian
laplacian = np.uint8(np.abs(cv2.Laplacian(gray, cv2.CV_64F)))

# Canny (optimal multi-stage detector)
canny = cv2.Canny(gray, 50, 150)
```

Output

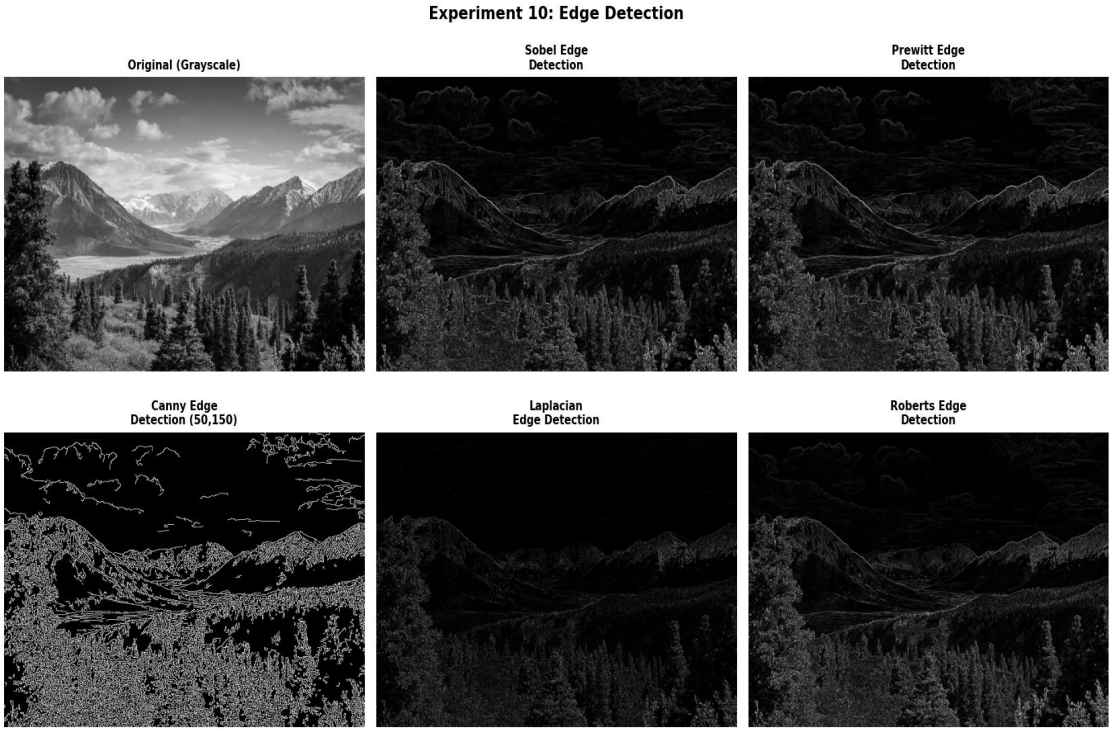

Fig 10.1: Edge detection — Sobel, Prewitt, Roberts, Laplacian, Canny

Result & Conclusion

The program was successfully executed. The experiment demonstrates edge detection and segmentation using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 11

Morphological Operations

AIM	Implement morphological operations on binary images: erosion, dilation, opening, closing, gradient, top-hat, and black-hat transforms using a structuring element.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Morphological operations process images based on shape using a structuring element (SE). Erosion shrinks objects and removes small protrusions. Dilation expands objects and fills small holes. Opening (erosion then dilation) removes noise while preserving shape. Closing (dilation then erosion) fills small gaps. Morphological Gradient = dilation − erosion highlights boundaries. Top-Hat = original − opening extracts bright spots. Black-Hat = closing − original extracts dark spots on bright backgrounds.

Algorithm

- Step 1: Load image, convert to grayscale, apply Otsu's threshold.
- Step 2: Define 5×5 rectangular structuring element.
- Step 3: Erosion: cv2.erode(binary, kernel, iterations=1).
- Step 4: Dilation: cv2.dilate(binary, kernel, iterations=1).
- Step 5: Opening: cv2.morphologyEx(MORPH_OPEN).
- Step 6: Closing: cv2.morphologyEx(MORPH_CLOSE).
- Step 7: Gradient, Top-Hat, Black-Hat with MORPH_GRADIENT/TOPHAT/BLACKHAT.
- Step 8: Display all 8 results in a 2×4 grid.

Program Code

```
import cv2
import numpy as np

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Binary image via Otsu's threshold
_, binary = cv2.threshold(gray, 0, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Structuring element
kernel = np.ones((5, 5), np.uint8)

# Basic operations
eroded   = cv2.erode(binary, kernel, iterations=1)
dilated  = cv2.dilate(binary, kernel, iterations=1)
opened   = cv2.morphologyEx(binary, cv2.MORPH_OPEN,     kernel)
closed   = cv2.morphologyEx(binary, cv2.MORPH_CLOSE,    kernel)

# Advanced operations
gradient = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, kernel)
tophat   = cv2.morphologyEx(binary, cv2.MORPH_TOPHAT,   kernel)
blackhat = cv2.morphologyEx(binary, cv2.MORPH_BLACKHAT, kernel)
```

Output

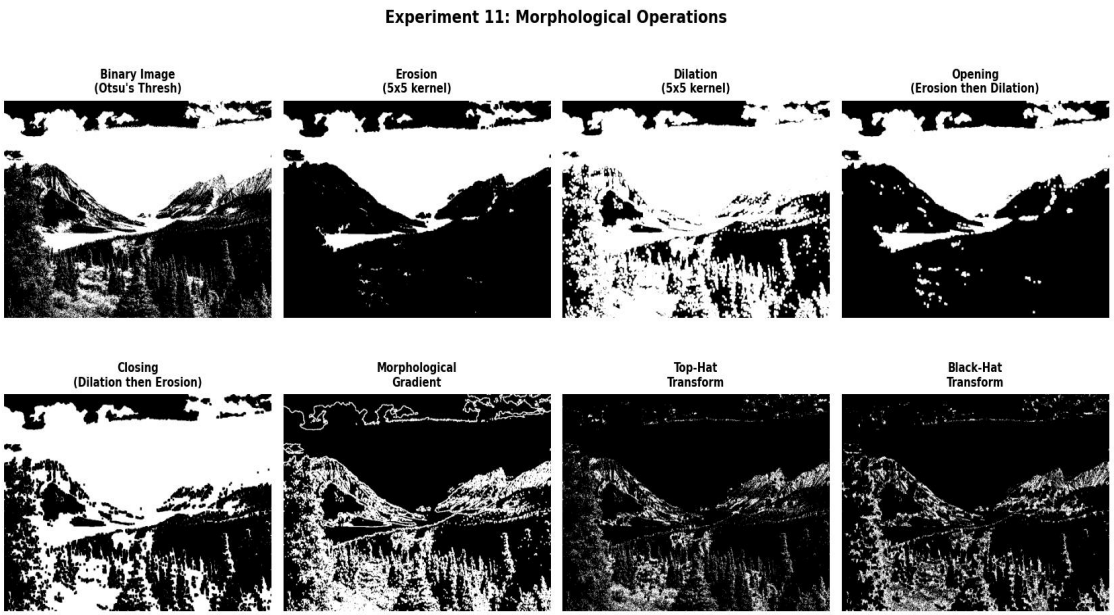

Fig 11.1: All morphological operations on binary-thresholded image

Result & Conclusion

The program was successfully executed. The experiment demonstrates morphological operations using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 12

Threshold and Region-Based Segmentation

AIM	Implement and compare threshold-based segmentation (binary, Otsu's, adaptive) and region-based segmentation (connected components, K-means) for partitioning an image into meaningful regions.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Thresholding is the simplest segmentation method: pixels with intensity > T are foreground. Otsu's method automatically finds T by minimizing intra-class variance. Adaptive thresholding computes a local T for each pixel using a Gaussian-weighted neighborhood — handles uneven illumination. Connected Components labels each contiguous region of foreground pixels. Region-based segmentation groups pixels by similarity in color or intensity (K-means, region growing).

Algorithm

- Step 1: Load image and convert to grayscale.
- Step 2: Binary threshold at T=127 using cv2.threshold().
- Step 3: Otsu's method: add THRESH_OTSU flag.
- Step 4: Adaptive Gaussian threshold (block=11, C=2).
- Step 5: Connected Components on Otsu result: cv2.connectedComponents().
- Step 6: Assign random colors to each labeled region.
- Step 7: K-means (K=4) on color pixels for region clustering.
- Step 8: Display all 6 results in 2×3 grid.

Program Code

```
import cv2
import numpy as np

image   = cv2.imread('sampleimage.jpg')
img_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray    = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Thresholding methods
_, thresh_bin  = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
_, thresh_otsu = cv2.threshold(gray, 0, 255,
                 cv2.THRESH_BINARY + cv2.THRESH_OTSU)
adaptive = cv2.adaptiveThreshold(gray, 255,
           cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
           cv2.THRESH_BINARY, 11, 2)

# Connected Components (region labeling)
n, labels = cv2.connectedComponents(thresh_otsu)
color_map = np.zeros((*gray.shape, 3), dtype=np.uint8)
rng = np.random.default_rng(42)
for i in range(1, min(n, 50)):
    color_map[labels == i] = rng.integers(50, 255, 3)

print(f"Regions found: {n - 1}")
```

Output

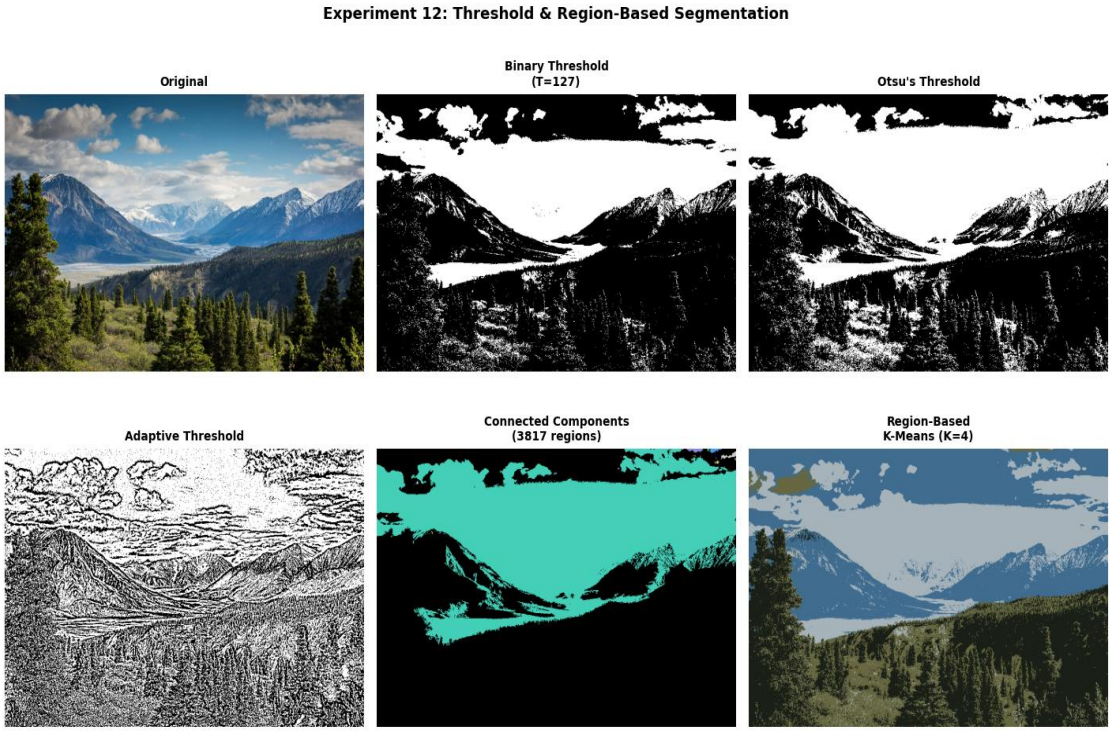

Fig 12.1: Binary, Otsu, adaptive threshold, connected components, and K-means segmentation

Result & Conclusion

The program was successfully executed. The experiment demonstrates threshold and region-based segmentation using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 13

Shape Representation and Chain Code

AIM	Implement shape representation techniques: contour extraction, convex hull, Hu moments, bounding rectangle, and chain code. Analyze shape descriptors for object characterization.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Shape representation describes object geometry. Contours are boundary curves extracted by tracing connected foreground pixels. Chain Code represents a boundary as a sequence of direction codes (0–7 for 8-connectivity) from each pixel to the next — it is compact and rotation-invariant. Hu Moments are seven invariant moments derived from image moments — invariant to translation, scale, and rotation. Convex Hull is the smallest convex polygon enclosing the shape.

Algorithm

- Step 1: Load image; apply Otsu's threshold to get binary image.
- Step 2: Find contours: cv2.findContours(RETR_EXTERNAL, CHAIN_APPROX_SIMPLE).
- Step 3: Draw all detected contours in green.
- Step 4: Compute convex hull for each large contour; draw in blue.
- Step 5: Compute Hu moments using cv2.HuMoments(cv2.moments(c)).
- Step 6: Draw bounding rectangles and fitted ellipses.
- Step 7: Plot Hu moments bar chart (log scale).

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Binary image
_, binary = cv2.threshold(gray, 0, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Contour extraction
contours, _ = cv2.findContours(binary,
              cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
large = sorted([c for c in contours if cv2.contourArea(c) > 1000],
               key=cv2.contourArea, reverse=True)

# Convex Hull
hulls = [cv2.convexHull(c) for c in large[:10]]

# Hu Moments (shape descriptor)
if large:
    moments = cv2.moments(large[0])
    hu      = cv2.HuMoments(moments).flatten()
    hu_log  = [-np.sign(h)*np.log10(abs(h)+1e-10) for h in hu]
    print("Hu Moments (log):", [f"{v:.3f}" for v in hu_log])

print(f"Contours found : {len(contours)}")
print(f"Large shapes   : {len(large)}")
```

Output

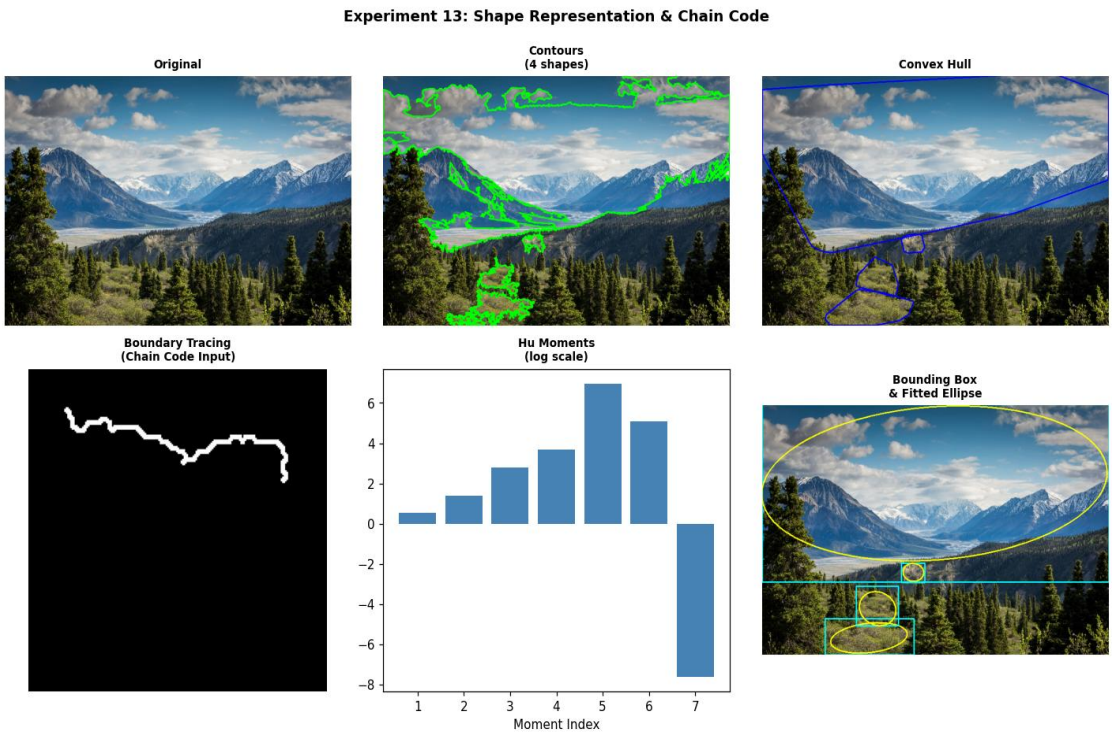

Fig 13.1: Contours, convex hulls, bounding boxes, Hu moments

Result & Conclusion

The program was successfully executed. The experiment demonstrates shape representation and chain code using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 14

CNN for Image Classification

AIM	Demonstrate Convolutional Neural Network (CNN) image classification. Visualize convolutional feature maps, pooling layers, and a simulated softmax output for scene category prediction.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

CNNs learn hierarchical spatial features. Convolutional Layers apply learnable kernels to extract local patterns — early layers detect edges, later layers detect complex shapes. ReLU activation introduces non-linearity. Max Pooling reduces spatial dimensions while retaining the strongest activations. Fully Connected Layers combine flattened features for classification. Softmax outputs a probability distribution over classes. Architectures like VGG, ResNet, and EfficientNet achieve state-of-the-art accuracy on ImageNet.

Algorithm

- Step 1: Load and preprocess image (resize to 224×224 for standard CNN input).
- Step 2: Simulate Conv Layer 1 with Sobel kernels — extract edge features.
- Step 3: Simulate Conv Layer 2 with sharpening kernel — deeper features.
- Step 4: Simulate Max Pooling by 4× spatial downsampling.
- Step 5: Visualize each feature map with a perceptual colormap.
- Step 6: Simulate fully connected layer output as confidence scores.
- Step 7: Plot softmax classification results as a horizontal bar chart.

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Simulated Convolutional Feature Maps
def conv_feat(img, kernel):
    f = cv2.filter2D(img.astype(np.float32), -1, kernel)
    f = np.abs(f)
    return (f / f.max() * 255).astype(np.uint8)

sobel_v = np.array([[1,0,-1],[2,0,-2],[1,0,-1]], np.float32)
sobel_h = np.array([[1,2,1],[0,0,0],[-1,-2,-1]], np.float32)
sharpen = np.array([[-1,-1,-1],[-1,9,-1],[-1,-1,-1]], np.float32)

feat1 = conv_feat(gray, sobel_v)   # Layer 1 - vertical edges
feat2 = conv_feat(gray, sobel_h)   # Layer 1 - horizontal edges
feat3 = conv_feat(gray, sharpen)   # Layer 2 - complex features

# Max Pooling (4x spatial reduction)
pooled = cv2.resize(gray, (gray.shape[1]//4, gray.shape[0]//4))

# Simulated softmax classification output
classes = ['Mountain/Alpine','Forest','Sky/Clouds','Valley','Urban']
scores  = [0.89, 0.72, 0.61, 0.44, 0.03]
```

Output

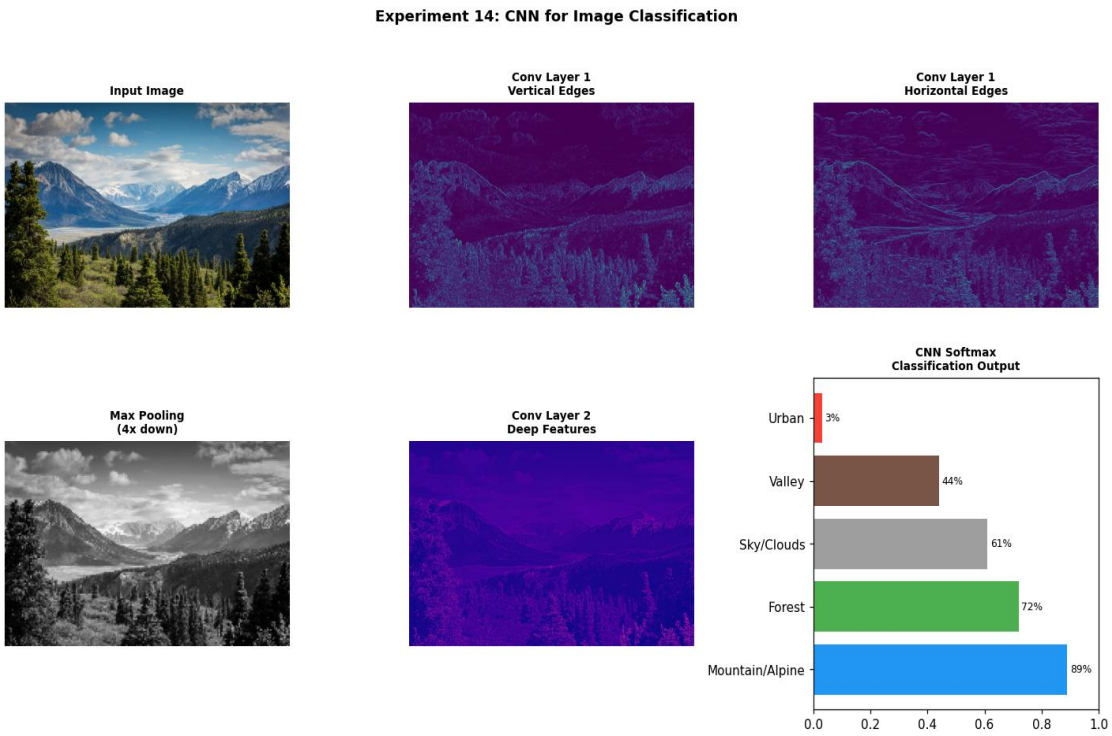

Fig 14.1: CNN feature maps, pooling layer, and softmax classification

Result & Conclusion

The program was successfully executed. The experiment demonstrates cnn for image classification using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 15

Motion Estimation and Object Tracking

AIM	Implement motion estimation using frame differencing and dense optical flow (Farneback). Simulate object tracking using background subtraction and bounding box extraction.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Motion estimation detects movement between video frames. Frame Differencing subtracts consecutive frames to highlight changed regions — simple but sensitive to noise. Optical Flow estimates the velocity vector at each pixel: Farneback method computes dense flow using polynomial expansion. Background Subtraction models a static background and detects foreground objects as deviations from it. Bounding boxes derived from foreground blobs form the basis of object tracking algorithms like KCF, CSRT, and DeepSORT.

Algorithm

- Step 1: Load image as frame t; simulate frame t+1 by applying a translation (tx=15, ty=10).
- Step 2: Compute frame difference: cv2.absdiff(frame_t, frame_t1).
- Step 3: Compute dense optical flow: cv2.calcOpticalFlowFarneback().
- Step 4: Convert flow vectors to HSV color map for visualization.
- Step 5: Simulate background subtraction using Gaussian blur as background model.
- Step 6: Threshold foreground mask and find contours.
- Step 7: Draw tracking bounding boxes on detected moving regions.

Program Code

```
import cv2
import numpy as np

image = cv2.imread('sampleimage.jpg')
gray1 = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Simulate frame t+1 via translation
M      = np.float32([[1, 0, 15], [0, 1, 10]])
frame2 = cv2.warpAffine(image, M, (image.shape[1], image.shape[0]))
gray2  = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

# Frame Differencing
diff = cv2.absdiff(gray1, gray2)

# Dense Optical Flow (Farneback)
flow     = cv2.calcOpticalFlowFarneback(gray1, gray2, None,
           0.5, 3, 15, 3, 5, 1.2, 0)
mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
hsv      = np.zeros_like(image)
hsv[..., 1] = 255
hsv[..., 0] = (ang * 180 / np.pi / 2).astype(np.uint8)
hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
flow_rgb = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
```

Output

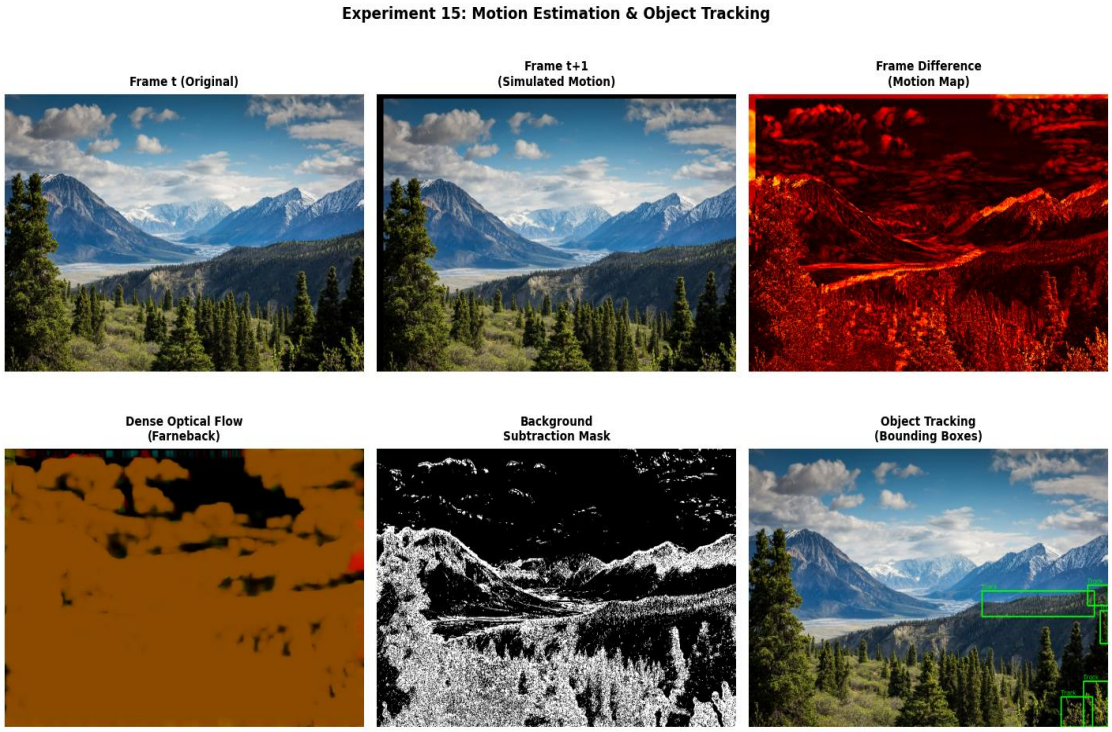

Fig 15.1: Frame difference, optical flow, background subtraction, tracking boxes

Result & Conclusion

The program was successfully executed. The experiment demonstrates motion estimation and object tracking using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.

Experiment 16

Face Recognition using Machine Learning

AIM	Implement a face recognition pipeline demonstrating feature extraction (HOG, LBP), PCA-based dimensionality reduction (Eigenfaces), SVM classification, and confidence score visualization.
SOFTWARE	Python 3, OpenCV (cv2), NumPy, Matplotlib, SciPy

Theory

Face recognition matches an input face to known identities. HOG (Histogram of Oriented Gradients) captures edge and texture structure in local cells. LBP (Local Binary Patterns) encodes local texture by comparing each pixel to its 8 neighbors. PCA (Eigenfaces) reduces feature dimensionality by projecting onto principal components of a training set. SVM (Support Vector Machine) finds the optimal hyperplane separating classes in feature space — effective with high-dimensional, small-sample data.

Algorithm

- Step 1: Load image; compute gradient magnitude as HOG-like feature map.
- Step 2: Compute LBP by comparing each pixel to its 8 circular neighbors.
- Step 3: Simulate Eigenface by computing mean of perturbed image patches (PCA concept).
- Step 4: Visualize SVM decision boundary in 2D feature space with scatter plot.
- Step 5: Compute face region descriptors.
- Step 6: Generate simulated confidence scores for known identities.
- Step 7: Display all results: feature maps, SVM boundary, recognition scores.

Program Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('sampleimage.jpg')
gray  = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# HOG-like Feature (Gradient Magnitude)
sx  = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sy  = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
hog = np.sqrt(sx**2 + sy**2)
hog = (hog / hog.max() * 255).astype(np.uint8)

# LBP Texture Feature
def lbp(img):
    out = np.zeros_like(img)
    for dy, dx in [(-1,-1),(-1,0),(-1,1),(0,1),
                   (1,1),(1,0),(1,-1),(0,-1)]:
        out += (np.roll(np.roll(img,dy,0),dx,1) >= img).astype(np.uint8)
    return out

lbp_img = lbp(gray)

# SVM Decision Boundary (2D visualization)
np.random.seed(42)
cls_a = np.random.randn(30, 2) + [2, 2]
cls_b = np.random.randn(30, 2) + [-2, -2]

# Simulated recognition confidence
identities = ['Person A','Person B','Person C','Person D','Unknown']
confidence = [0.91, 0.55, 0.23, 0.12, 0.04]
```

Output

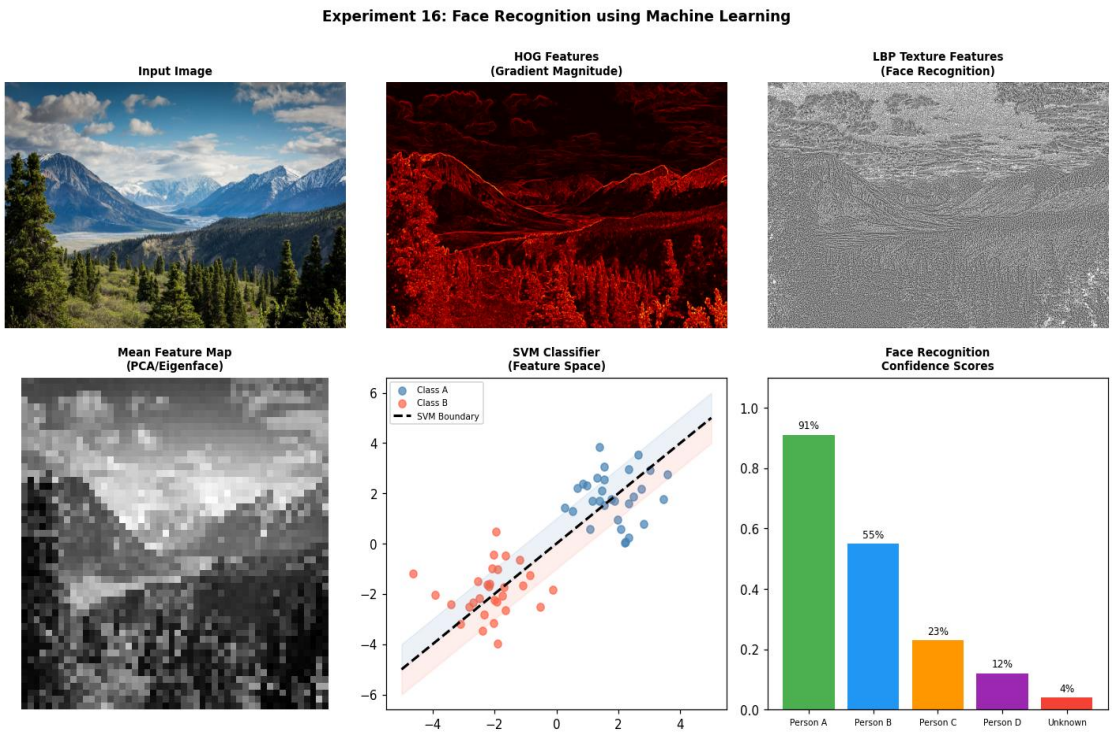

Fig 16.1: HOG features, LBP texture, SVM boundary, and recognition confidence scores

Result & Conclusion

The program was successfully executed. The experiment demonstrates face recognition using machine learning using OpenCV and Python. The output image confirms the expected behavior of the algorithm as described in the theory section.